NAME : NITHIN.V

ROLL.NO : 2403A52355

BATCH.NO : 13(AIML)

STEP 1 — Import Required Libraries

```
# Data handling libraries
import pandas as pd          # For handling tabular data
import numpy as np           # For numerical operations

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns

# Hugging Face libraries
from datasets import load_dataset        # To load datasets
from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    Trainer,
    TrainingArguments
)

# PyTorch
import torch

# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix,
    classification_report
)
```

STEP 2 — Load and Explore Dataset

```
# Load IMDb dataset
dataset = load_dataset("imdb")
# Display dataset structure
print(dataset)
# Show sample record
print(dataset["train"][0])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:        7.81k/?  [00:00<00:00, 560kB/s]

plain_text/train-00000-of-00001.parquet: 100%                    21.0M/21.0M  [00:00<00:00, 105MB/s]

plain_text/test-00000-of-00001.parquet: 100%                    20.5M/20.5M  [00:00<00:00, 102MB/s]

plain_text/unsupervised-00000-of-00001.p(…): 100%                    42.0M/42.0M  [00:00<00:00, 68.5MB/s]

Generating train split: 100%                    25000/25000  [00:00<00:00, 87638.86 examples/s]

Generating test split: 100%                    25000/25000  [00:00<00:00, 108171.91 examples/s]

Generating unsupervised split: 100%                    50000/50000  [00:00<00:00, 129251.63 examples/s]

DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was fir
```

```
print("Columns:", dataset["train"].column_names)
```

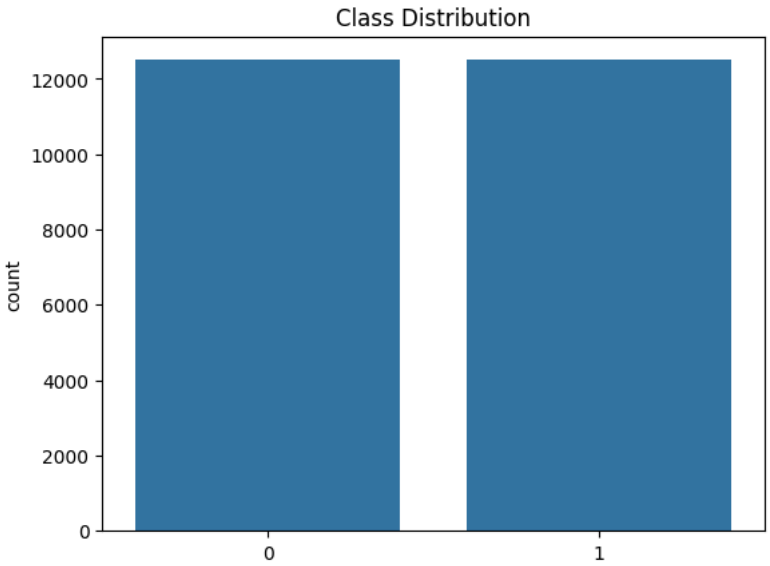
```
Columns: ['text', 'label']
```

```
train_labels = dataset["train"]["label"]

# Count class distribution
unique, counts = np.unique(train_labels, return_counts=True)
print(dict(zip(unique, counts)))

# Plot class distribution
sns.countplot(x=train_labels)
plt.title("Class Distribution")
plt.show()
```

```
{np.int64(0): np.int64(12500), np.int64(1): np.int64(12500)}
```



STEP 3 — Text Tokenization and Preprocessing

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 2.98kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 545kB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 1.82MB/s]
```

```
def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",      # Pad to fixed length
        truncation=True,           # Truncate long sequences
        max_length=128
    )
```

```
tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

```
Map: 100%  25000/25000 [00:18<00:00, 1330.64 examples/s]
Map: 100%  25000/25000 [00:17<00:00, 1442.33 examples/s]
Map: 100%  50000/50000 [00:49<00:00, 818.73 examples/s]
```

```
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
tokenized_datasets.set_format(
    "torch",
    columns=["input_ids", "attention_mask", "labels"]
)
```

STEP 5 — Train–Test Split

```
train_dataset = tokenized_datasets["train"]
test_dataset = tokenized_datasets["test"]
```

STEP 6 — Load Pre-trained Transformer Model

```
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
config.json: 100%  570/570 [00:00<00:00, 48.5kB/s]

model.safetensors: 100%  440M/440M [00:03<00:00, 192MB/s]

Loading weights: 100%  199/199 [00:00<00:00, 528.14it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
-------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |
cls.seq_relationship.bias                  | UNEXPECTED |
cls.predictions.transform.dense.weight     | UNEXPECTED |
cls.predictions.transform.dense.bias       | UNEXPECTED |
cls.seq_relationship.weight                | UNEXPECTED |
cls.predictions.bias                       | UNEXPECTED |
classifier.bias                            | MISSING    |
classifier.weight                          | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
```

STEP 7 — Define Training Configuration

```
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average='binary'
    )

    acc = accuracy_score(labels, preds)

    return {
        'accuracy': acc,
        'precision': precision,
        'recall': recall,
        'f1': f1
    }
```

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=100
)
```

STEP 8 — Train the Model

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
trainer.train()
trainer.evaluate()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as tru
  super().__init__(loader)
```

[481/3126 2:29:37 < 13:46:12, 0.05 it/s, Epoch 0.31/2]

Epoch	Training Loss	Validation Loss

STEP 9 — Model Evaluation

```
predictions = trainer.predict(test_dataset)

y_true = predictions.label_ids
y_pred = predictions.predictions.argmax(-1)

print(classification_report(y_true, y_pred))
```

[119/3126 31:33 < 13:31:04, 0.06 it/s, Epoch 0.08/2]

Epoch	Training Loss	Validation Loss

[722/1563 59:52 < 1:09:50, 0.20 it/s]

```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

STEP 11 — Result Analysis

1. BERT uses contextual embeddings unlike traditional TF-IDF.
2. Bidirectional understanding improves sentiment detection.
3. Performance usually exceeds Naive Bayes and CNN.
4. Pretrained embeddings reduce need for large labeled data.
5. Transformer captures long-range dependencies.
6. However, training time is high.
7. Requires GPU for efficiency.
8. Memory consumption is large.
9. Hyperparameter tuning can further improve results.
10. Using larger models like RoBERTa may boost accuracy.